# FPGA Lab Coding Standards and Engineering Methodology

This document defines the standard engineering practices, RTL coding methodologies, Tcl scripting conventions, and project organization rules to be followed within the FPGA laboratory. The purpose of these standards is to create a professional engineering culture, improve readability, reduce debugging effort, and ensure long-term maintainability and scalability of FPGA projects.

## 1. RTL Naming Rules

- All RTL modules and signals must follow meaningful and consistent naming conventions.
- Inputs should use prefix i_ (Example: i_clk, i_rst_n, i_data).
- Outputs should use prefix o_ (Example: o_valid, o_ready).
- Bidirectional ports should use prefix io_ (Example: io_gpio).
- Registers should use prefix r_ (Example: r_counter, r_state).
- Combinational signals should use prefix w_ (Example: w_sum, w_enable).
- Parameters must use uppercase naming (Example: DATA_WIDTH, FIFO_DEPTH).
- FSM states should use uppercase localparams (Example: IDLE, READ, WRITE).
- Module names must use lowercase_with_underscores style.
- RTL file names must match module names.
  For example

```
Control_Unit cu(.i_op(i_instr), .o_result(w_result));
Data_path dp(.i_result(w_result), .o_sum(o_signal));
```

  in this you can see there are two module instantiations where "o_result" is output from cu and is given as input to dp as "i_result" and both are connected through wire "w_result". Looks very easy if the rules are followed or else would have been difficult as all would end with same name "result".

## 2. Tcl Naming Rules

- All Tcl scripts must remain modular, reusable, and readable.
- Variables should use descriptive snake_case naming.
- Constants should use uppercase naming.
- Procedure names should describe actions such as run_synthesis or generate_reports.
- Separate Tcl scripts based on functionality.

## 3. Directory Structure Rules

- All projects must follow a standard directory structure.
- rtl/ contains RTL source files.
- constraints/ contains XDC files.
- scripts/ contains Tcl automation scripts.
- tb/ contains testbench files.
- sim/ contains simulation files.
- reports/ contains timing and utilization reports.
- build/ contains build checkpoints.
- logs/ contains execution logs.
- docs/ contains documentation and notes.
- Random and inconsistent folders are not allowed.

## 4. Reset Policy

- Reset polarity must always be visible in naming.
- Active-low resets should use i_rst_n.
- Synchronous reset methodology is preferred unless asynchronous reset is required.
- Only essential registers should be reset to reduce routing overhead.

## 5. Clock Naming Policy

- Clock domains must always be identifiable.
- Single-clock systems should use i_clk.
- Multi-clock systems should use domain-aware names such as i_clk_axi or i_clk_ddr.
- Clock names in XDC must remain meaningful.
- CDC paths must be documented properly.

## 6. FSM Style Guide

- FSMs must follow a clean and maintainable coding style.
- Use r_state for current state and w_next_state for next-state logic.
- FSM states must be uppercase localparams.
- Separate sequential and combinational logic blocks.
- FSM transitions should remain easy to trace and debug.

## 7. Pipeline Naming Rules

- Pipeline stages must be clearly identifiable.
- Use names such as r_data_s1, r_data_s2, and r_data_s3.
- Pipeline stage visibility improves timing analysis and debugging.

## 8. RTL Coding Guidelines

- Prefer synchronous design methodology.
- Use nonblocking assignments in sequential logic.
- Use blocking assignments in combinational logic.
- Avoid unintended latch generation.
- Use parameterization to improve module reusability.
- Maintain clean hierarchy and modular design structure.
- Design with timing closure awareness from the beginning.

## 9. Commenting Standards

- Every module must contain a descriptive module header.
- Complex interfaces and signals must be documented.
- FSM functionality and transitions should be commented.
- Important constraints must contain explanations.
- Comments should explain engineering intent, not obvious syntax.

## 10. Report Analysis Culture

- Students must inspect reports rather than only checking bitstream generation.
- Always analyze WNS, TNS, hold violations, and utilization.
- Critical timing paths should be studied carefully.
- QoR tracking should become part of development culture.

## 11. Git and Version Control Discipline

- Use meaningful commit messages.
- Use feature branches for development.
- Avoid unstable code in the main branch.
- Maintain version history for architectural changes.

## 12. Engineering Philosophy

- The objective is to establish a disciplined engineering culture within the laboratory.
- Students should learn reproducibility, automation, timing awareness, and maintainable RTL design.
- The focus should remain on methodology and engineering discipline rather than tool usage alone.

These standards are expected to evolve over time as the laboratory grows and projects become more advanced. However, the core philosophy of disciplined engineering, readability, automation, maintainability, and reproducibility must always remain unchanged.